

Gestione del rumore nel Liver Patient Dataset

2 luglio 2026

Indice

1	Obiettivo dell'esperimento	2
2	Preparazione dei dati	2
2.1	Suddivisione del dataset	2
2.2	Significato del target	2
3	Selezione dei gruppi di feature	3
4	Modalità di corruzione	3
4.1	Feature continue	3
4.2	Feature categoriche	3
4.3	Gestione dei seed	3
5	Rete neurale	4
5.1	Scaling	4
5.2	Gestione dello sbilanciamento	4
5.3	Ensemble e addestramento	4
6	Metriche	5
7	Media e deviazione standard	5
7.1	Perché sono necessarie	5
8	Decision Tree	6
9	Support Vector Machine	6
10	Interpretazione dei risultati	6
11	Riproducibilità e procedura di esecuzione	7
12	Conclusioni	7

1 Obiettivo dell'esperimento

L'esperimento misura quanto le prestazioni di un classificatore cambino quando una parte crescente del training set viene corrotta. Il rumore viene applicato esclusivamente alle feature del training set; validation set e test set rimangono puliti. Questa scelta simula una situazione nella quale i dati disponibili per l'addestramento contengono errori di misura, mentre la valutazione viene eseguita su osservazioni non corrotte.

La percentuale di rumore assume i valori

$$\rho \in \{0, 0.1, 0.2, \dots, 0.9, 1\}.$$

Il valore ρ indica la frazione di righe modificate, non l'intensità della singola perturbazione. Per esempio, $\rho = 0.8$ significa che viene selezionato l'80% delle righe del training set per ciascuna feature analizzata.

Il livello $\rho = 0$ costituisce la baseline pulita. Il livello $\rho = 1$ indica che tutte le righe vengono perturbate, ma non che l'informazione originale venga completamente eliminata.

2 Preparazione dei dati

2.1 Suddivisione del dataset

Lo split viene eseguito una sola volta prima di scorrere i livelli di rumore. L'uso dello stesso split rende confrontabili i risultati ottenuti per valori differenti di ρ .

Per la rete neurale viene usata la suddivisione:

- 30% test set;
- il restante 70% viene ulteriormente diviso in training e validation;
- con `validation_size=0.2`, le proporzioni complessive sono circa 56% training, 14% validation e 30% test.

Gli split sono stratificati, quindi mantengono approssimativamente la distribuzione delle classi. Il test set non viene usato per addestrare i modelli né per scegliere il numero di epoche.

Negli esperimenti progressivi degli alberi è presente la stessa separazione train-validation-test: la configurazione viene scelta sul validation set e misurata successivamente sul test set. La SVM usa invece uno split train-test perché i suoi iperparametri sono attualmente fissati.

2.2 Significato del target

Nel preprocessing generale, `cat.codes` assegna normalmente:

Valore originale	Codice iniziale
Liver Disease	0
No Liver Disease	1

All'interno degli esperimenti noise di rete neurale e SVM il target viene invertito localmente:

$$y_{\text{noise}} = 1 - y_{\text{encoded}}.$$

Di conseguenza, durante questi test la classe positiva 1 rappresenta *Liver Disease*, mentre la classe 0 rappresenta *No Liver Disease*. La modifica è locale e non altera le sezioni precedenti del notebook.

È importante riavviare il kernel ed eseguire le celle in ordine: un oggetto `df_encoded` rimasto in memoria con una codifica precedente può invertire nuovamente il significato delle metriche.

3 Selezione dei gruppi di feature

Le feature vengono ordinate attraverso la Mutual Information rispetto al target. Negli esperimenti aggiornati, la Mutual Information viene calcolata esclusivamente sul training set:

$$I(X_j; Y) = \sum_{x,y} p(x,y) \log \left(\frac{p(x,y)}{p(x)p(y)} \right).$$

Le feature ordinate vengono divise in due gruppi:

- **migliori**: metà con Mutual Information più elevata;
- **peggiori**: metà con Mutual Information più bassa.

Il calcolo sul solo training evita che test e validation influenzino la scelta delle feature da corrompere. I nomi presenti nelle variabili globali **migliori** e **peggiori** vengono quindi ignorati quando viene specificato il parametro **feature_group**; il gruppo viene ricostruito dentro la funzione dell'esperimento.

4 Modalità di corruzione

4.1 Feature continue

Per una feature continua X_j vengono selezionate senza reinserimento

$$n_\rho = \lfloor \rho n_{\text{train}} \rfloor$$

righe. A ogni valore selezionato viene aggiunto rumore gaussiano:

$$\tilde{x}_{ij} = x_{ij} + \epsilon_{ij}, \quad \epsilon_{ij} \sim \mathcal{N}(0, \sigma_j^2),$$

con

$$\sigma_j = \lambda s_j.$$

s_j è la deviazione standard della feature nel training set e λ è il parametro **noise_level**, posto a 0.5 nelle chiamate correnti. Pertanto anche al 100% di righe corrotte il valore originale rimane presente; viene aggiunta una perturbazione con deviazione standard pari a metà della deviazione standard della colonna. Non è quindi obbligatorio osservare un crollo delle metriche a $\rho = 1$.

4.2 Feature categoriche

Per una feature categorica viene selezionata la frazione ρ delle righe. Il valore di ciascuna riga selezionata viene sostituito con un valore estratto dalle altre categorie osservate nella stessa colonna. Il nuovo valore è sempre differente da quello originale, quando sono disponibili almeno due categorie.

4.3 Gestione dei seed

Ogni colonna usa un seed distinto. Nella rete neurale il seed dipende anche dalla ripetizione:

$$s_{r,j} = s_0 + 1000r + j,$$

dove r identifica la ripetizione e j la feature. Per una stessa coppia (r, j) il seed non cambia tra i livelli di rumore. In questo modo:

- feature differenti non ricevono sequenze casuali identiche;
- il confronto tra livelli di rumore è accoppiato e meno influenzato da variazioni casuali estranee;
- ripetizioni differenti usano corruzioni indipendenti.

5 Rete neurale

5.1 Scaling

Dopo la corruzione viene costruito un `RobustScaler`. Lo scaler viene stimato esclusivamente sul training set relativo al livello corrente:

$$z_{ij} = \frac{x_{ij} - \text{mediana}(X_j)}{Q_{3,j} - Q_{1,j}}.$$

Lo stesso scaler trasforma validation e test set. Non viene mai eseguito `fit` su validation o test, evitando data leakage. Il target non viene scalato, perché deve rimanere binario.

5.2 Gestione dello sbilanciamento

I pesi delle classi sono calcolati sul training set con la regola bilanciata di scikit-learn:

$$w_c = \frac{n}{Kn_c},$$

dove n è il numero totale di esempi, $K = 2$ il numero di classi e n_c il numero di osservazioni della classe c . I pesi vengono passati a `model.fit`; gli errori sulla classe meno frequente contribuiscono quindi maggiormente alla loss.

Non viene effettuato undersampling. Il bootstrap mantiene mediamente lo sbilanciamento del campione originale, mentre `class_weight` interviene nella funzione di costo.

5.3 Ensemble e addestramento

L'architettura contiene due livelli nascosti densi da 32 e 16 neuroni con attivazione ReLU e un neurone di output sigmoidale. Per ogni ripetizione vengono addestrate tre reti su campioni bootstrap del training set.

Per il modello k della ripetizione r viene usato

$$s_{r,k} = s_0 + 1000r + k.$$

Il seed di uno stesso modello non dipende da ρ : inizializzazione e indici bootstrap rimangono confrontabili tra livelli di rumore. La validation loss viene controllata tramite `EarlyStopping`, con pazienza pari a 5 e ripristino dei pesi migliori.

Se $p_{r,k}(x)$ è la probabilità prodotta dal modello k , la probabilità ensemble è

$$\bar{p}_r(x) = \frac{1}{E} \sum_{k=1}^E p_{r,k}(x), \quad E = 3.$$

La classe finale è

$$\hat{y}_r(x) = \begin{cases} 1 & \text{se } \bar{p}_r(x) > 0.5, \\ 0 & \text{altrimenti.} \end{cases}$$

6 Metriche

Indicando con TP, TN, FP e FN gli elementi della matrice di confusione, vengono calcolate:

$$\begin{aligned}\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN}, \\ \text{Recall}_1 &= \frac{TP}{TP + FN}, \\ \text{Precision}_1 &= \frac{TP}{TP + FP}, \\ F1_1 &= 2 \frac{\text{Precision}_1 \text{Recall}_1}{\text{Precision}_1 + \text{Recall}_1}, \\ \text{Balanced Accuracy} &= \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).\end{aligned}$$

Accuracy può risultare elevata anche quando la classe minoritaria viene riconosciuta male. Balanced accuracy attribuisce la stessa importanza alle due classi. Precision indica quanti dei pazienti classificati come malati sono effettivamente malati; recall indica quanti pazienti malati vengono riconosciuti. F1 rappresenta un compromesso tra precision e recall.

7 Media e deviazione standard

7.1 Perché sono necessarie

L'addestramento della rete neurale è stocastico. Le prestazioni dipendono da inizializzazione dei pesi, bootstrap, ordine dei dati e realizzazione del rumore. Un singolo valore per livello non permette di distinguere l'effetto del rumore dalla variabilità casuale.

Per questo motivo l'intero ensemble viene ripetuto $R = 5$ volte per ogni livello ρ . Ogni ripetizione produce una metrica $m_{\rho,r}$, con $r = 1, \dots, R$.

La media campionaria è

$$\bar{m}_\rho = \frac{1}{R} \sum_{r=1}^R m_{\rho,r}.$$

La deviazione standard campionaria è

$$s_\rho = \sqrt{\frac{1}{R-1} \sum_{r=1}^R (m_{\rho,r} - \bar{m}_\rho)^2}.$$

Nel codice viene usato `numpy.std(ddof=1)`, corrispondente al denominatore $R-1$. Se viene richiesta una sola ripetizione, la deviazione standard viene impostata a zero.

Il DataFrame finale contiene, per esempio:

Colonna	Significato
<code>accuracy</code>	media dell'accuracy
<code>accuracy_std</code>	deviazione standard dell'accuracy
<code>balanced_accuracy</code>	media della balanced accuracy
<code>balanced_accuracy_std</code>	relativa deviazione standard
<code>f1_class_1</code>	media dell'F1 della classe positiva
<code>f1_class_1_std</code>	relativa deviazione standard

Nel grafico il punto rappresenta $\overline{m}_\rho$ e la barra d'errore rappresenta $\overline{m}_\rho \pm s_\rho$. Le barre non sono intervalli di confidenza. Una deviazione standard elevata indica instabilità tra ripetizioni; barre sovrapposte suggeriscono che le differenze osservate potrebbero essere dovute alla variabilità, ma non costituiscono da sole un test statistico.

Poiché tutte le ripetizioni usano lo stesso test set, la deviazione standard misura soprattutto la variabilità dovuta a rumore e addestramento. Non misura completamente l'incertezza dovuta alla scelta del campione di test. Per includere anche tale componente servirebbero split ripetuti o cross-validation esterna.

8 Decision Tree

Negli esperimenti noise degli alberi vengono provate diverse profondità, valori di `ccp_alpha` e configurazioni di `class_weight`. Ogni configurazione produce un ensemble di tre alberi. Ciascun albero viene addestrato sui due fold di training di una suddivisione stratificata a tre fold, cioè su circa il 66% del training set.

La configurazione migliore viene selezionata sul validation set secondo la metrica indicata, attualmente `f1_class_1`. Solo dopo la selezione l'ensemble viene valutato sul test set.

Anche qui non viene effettuato undersampling: lo sbilanciamento viene gestito provando pesi differenti. La versione corrente restituisce un singolo risultato per livello di rumore; non calcola ancora media e deviazione standard su ripetizioni indipendenti.

9 Support Vector Machine

La SVM usa kernel RBF, $C = 1$, `gamma=scale` e `class_weight=balanced`. Per ciascun livello vengono costruiti tre modelli su campioni bootstrap. La predizione finale è ottenuta mediante voto di maggioranza sulle classi predette.

Il `RobustScaler` viene stimato sul training corrotto e applicato al test pulito. Questo passaggio è essenziale per il kernel RBF, perché distanze e parametro gamma sono sensibili alla scala delle feature.

Target, selezione tramite Mutual Information e seed distinti per colonna vengono gestiti localmente nella funzione noise. Come per gli alberi, la versione corrente produce un singolo valore per livello e non calcola ancora media e deviazione standard su più ensemble.

10 Interpretazione dei risultati

Non è necessario che le metriche diminuiscano monotonamente all'aumentare di ρ . Le ragioni principali sono:

- il rumore continuo è additivo e non elimina completamente il segnale;
- una perturbazione moderata può agire occasionalmente come regolarizzazione;
- il dataset è piccolo e le stime hanno variabilità elevata;
- scaling, pesi di classe ed ensemble possono ridurre la sensibilità alla corruzione;
- differenze inferiori alla variabilità mostrata dalle barre d'errore non devono essere sovrainterpretate.

Prima di interpretare un grafico è opportuno verificare la distribuzione del target nel test set. Con la convenzione usata dagli esperimenti NN e SVM ci si aspetta circa il 71% di classe 1. Precision, recall e accuracy devono essere compatibili con tale prevalenza; valori incompatibili indicano spesso che il kernel non è stato riavviato dopo una modifica della codifica.

11 Riproducibilità e procedura di esecuzione

Per ottenere risultati coerenti:

1. riavviare il kernel Jupyter;
2. eseguire il notebook dall'inizio, senza saltare celle che definiscono funzioni;
3. eseguire la cella che definisce la funzione noise del modello;
4. eseguire separatamente il gruppo `peggiori` e il gruppo `migliori`;
5. salvare DataFrame, grafici e configurazione dei seed;
6. aggiornare tabelle e figure della relazione, evitando di riutilizzare output prodotti da versioni precedenti del codice.

Con $R = 5$, $E = 3$ e 11 livelli di rumore, un singolo gruppo della rete neurale richiede

$$5 \times 3 \times 11 = 165$$

addestramenti. L'esecuzione di entrambi i gruppi richiede 330 addestramenti. Questo costo è la conseguenza necessaria della stima della variabilità tramite ripetizioni.

12 Conclusioni

Il test noise confronta modelli addestrati su versioni progressivamente corrotte dello stesso training set e valutati su dati puliti. La validità del confronto dipende da quattro condizioni: assenza di leakage, codifica coerente del target, scaling stimato solo sul training e controllo delle sorgenti di casualità.

Per la rete neurale, media e deviazione standard forniscono un risultato più affidabile del singolo esperimento. La media descrive la prestazione tipica, mentre la deviazione standard quantifica la stabilità rispetto alle realizzazioni casuali. Un eventuale miglioramento a livelli elevati di rumore deve essere considerato significativo solo se è ripetibile e sufficientemente grande rispetto alla variabilità osservata.